

Algoritmi di ricerca e strategie di ricerca non informata

Claudio Gallicchio, 2025/26
claudio.gallicchio@unipi.it

Riferimenti

AIMA Cap. 3

Sez. 3.3, 3.4



Algoritmi di ricerca

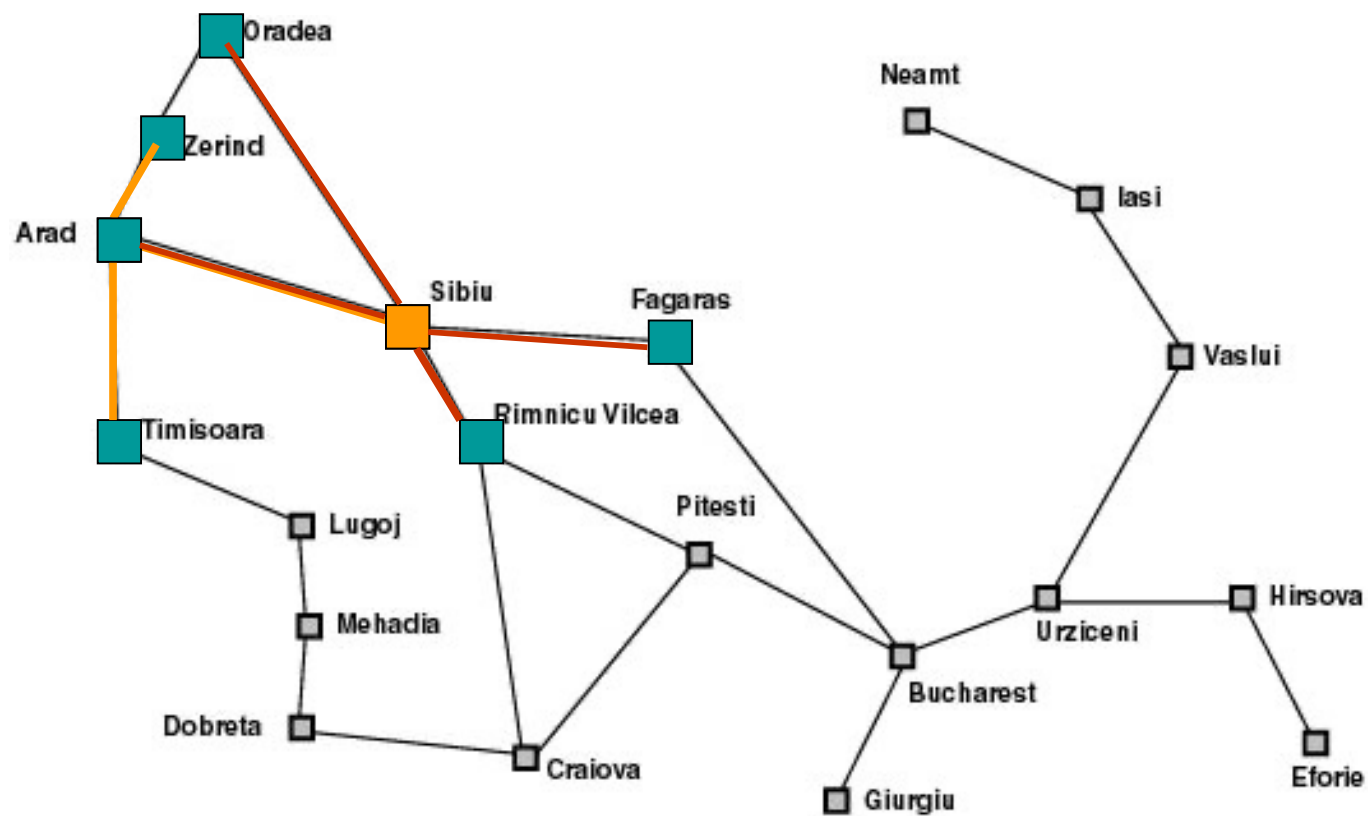
Problemi di ricerca recap

Un problema di ricerca può essere definito formalmente:

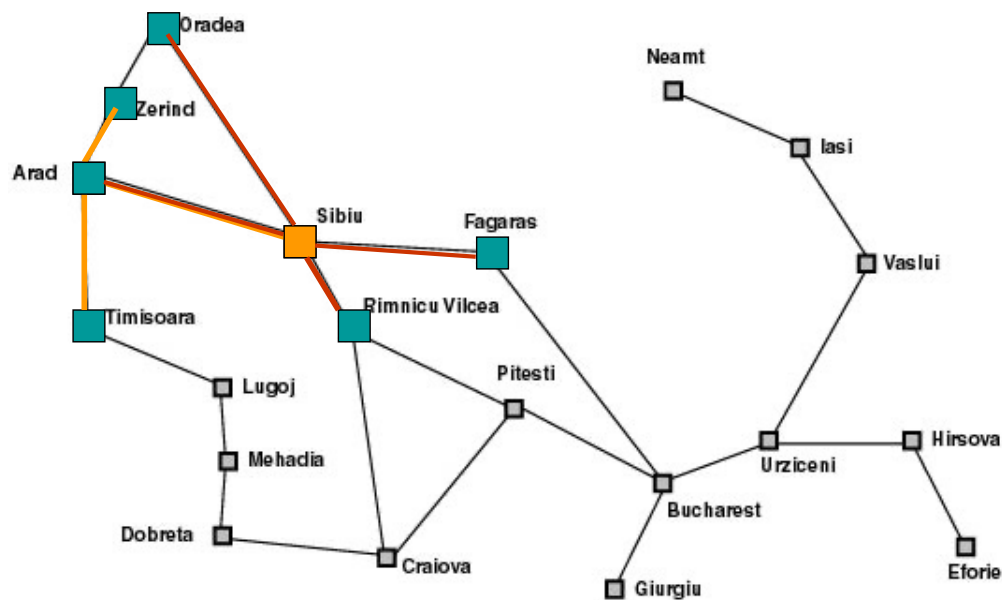
1. Spazio degli stati
2. Stato iniziale
3. Stato(/i) obiettivo
4. Azioni
5. Modello di transizione
6. Funzione costo

Un **algoritmo di ricerca** riceve in **input** un **problema di ricerca** e restituisce in **output** **una soluzione** (o una indicazione di fallimento).

Algoritmo di ricerca

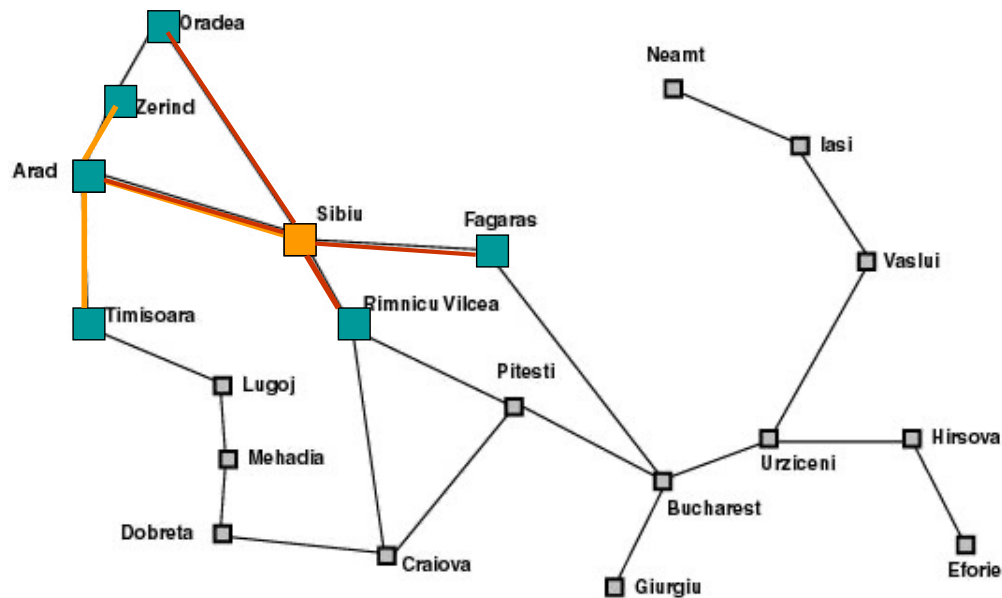


Algoritmo di ricerca



Consideriamo
algoritmi che
sovrappongono un
albero di ricerca al
grafo dello spazio
degli stati

Algoritmo di ricerca



Albero di ricerca

- ogni **nodo** corrisponde a uno **stato nello spazio degli stati**
- i **rami** corrispondono alle **azioni**
- la **radice** corrisponde allo **stato iniziale** del problema

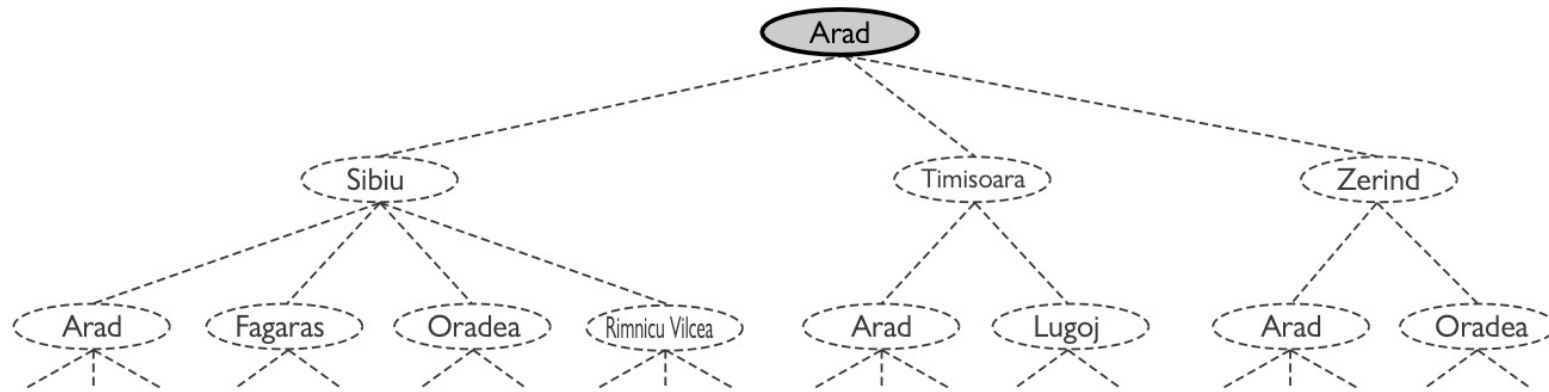
Spazio degli stati $\neq$ Albero di ricerca

Spazio degli stati

Descrive l'insieme (potenzialmente infinito) degli stati nel mondo e le azioni che consentono transizioni tra essi.

Albero di ricerca

- Descrive i cammini tra gli stati per raggiungere l'obiettivo.
- Possono esserci piu' cammini per raggiungere qualsiasi stato.
- Piu' nodi possono corrispondere allo stesso stato.
- Per ogni nodo, c'e' un solo cammino per tornare alla radice.

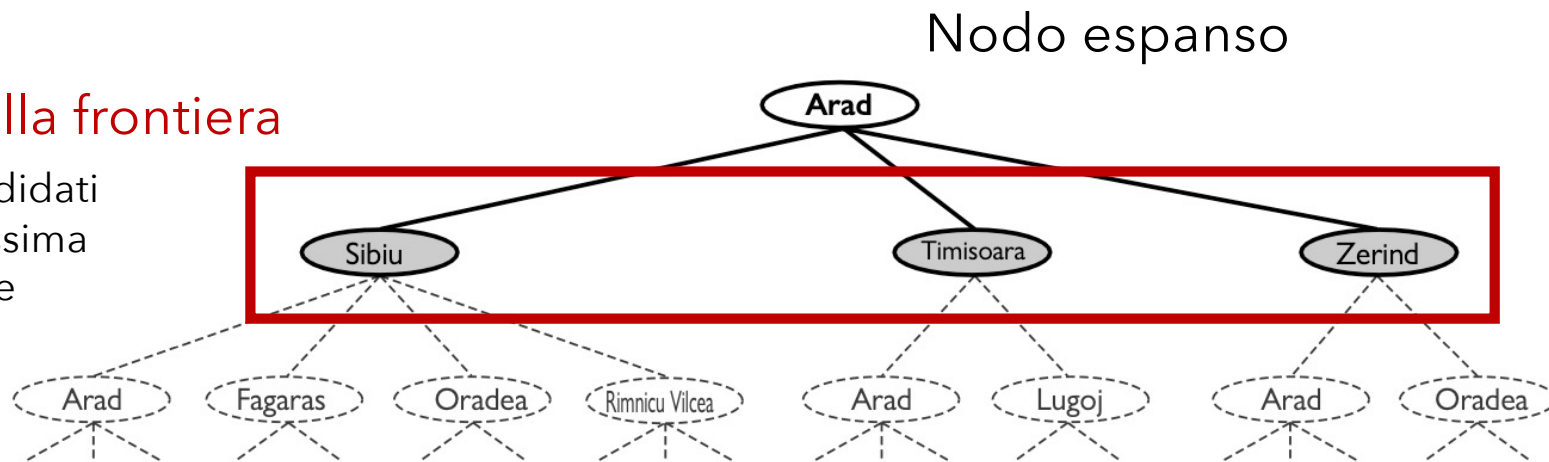


Espansione di un nodo.

- Consideriamo le azioni disponibili nello stato corrispondente.
- Generiamo un nuovo nodo (figlio, o successore) per ognuno degli stati risultati.
- Ciascuno dei nodi generati avra' il nodo espanso come padre.

Nodi nella frontiera

sono i candidati
per la prossima
espansione



Frontiera: insieme dei nodi generati ma non ancora espansi

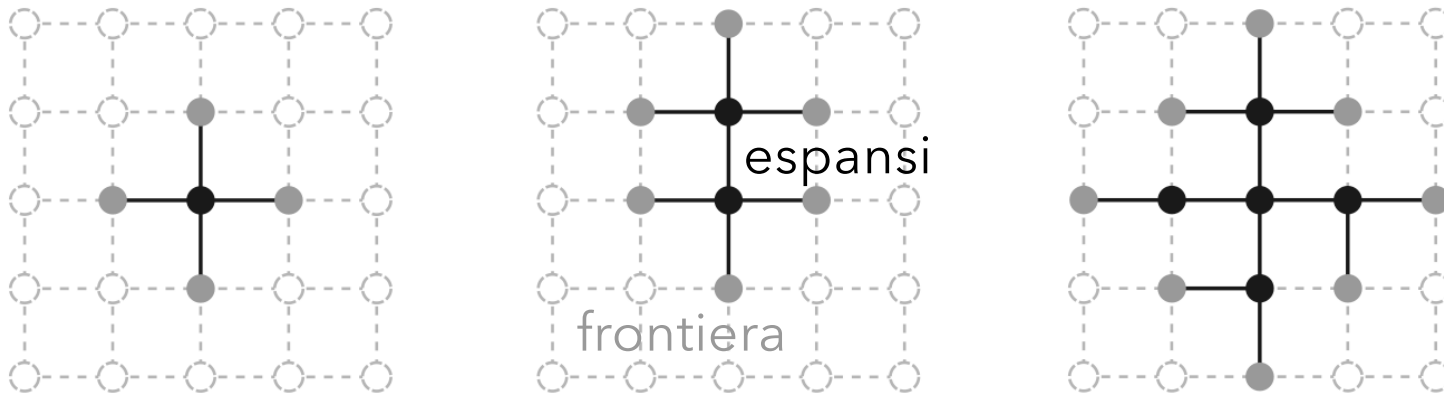
Stati raggiunti: stati del grafo per cui esiste un corrispondente nodo generato nell'albero di ricerca (anche se non ancora espanso).

Frontiera

Separa due regioni del grafo dello spazio degli stati:

- una regione interna in cui ogni nodo e' stato espanso
- una regione esterna di stati che non sono ancora stati raggiunti

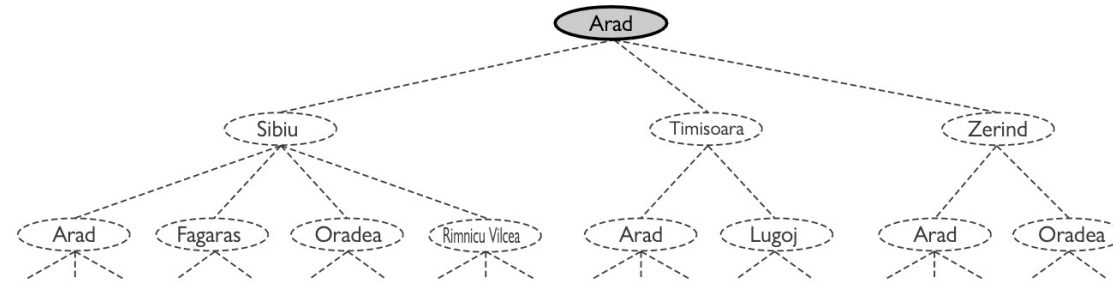
Frontiera



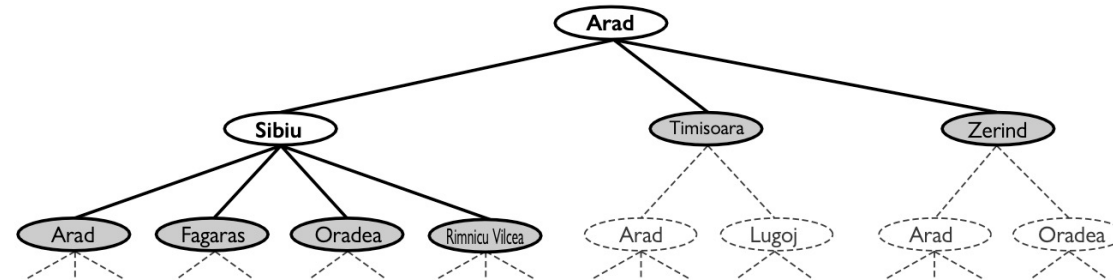
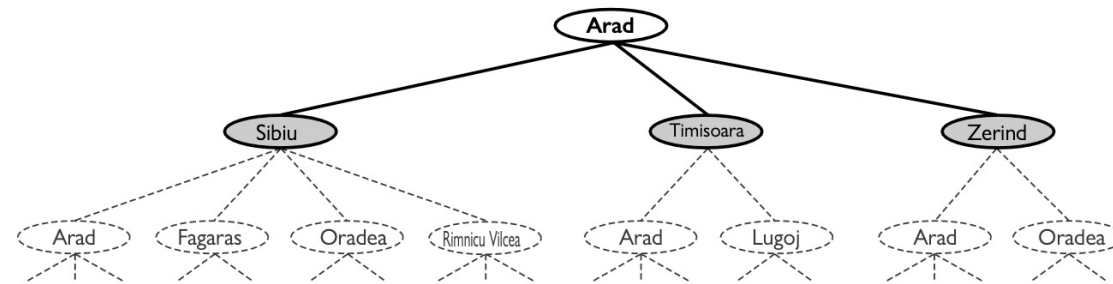
Separa due regioni del grafo dello spazio degli stati:

- una regione interna in cui ogni nodo e' stato espanso
- una regione esterna di stati che non sono ancora stati raggiunti

Esempio di alberi di ricerca per il problema della Romania.



La ricerca va avanti espandendo ogni volta un nodo della frontiera.



Come scegliamo quale nodo
della frontiera espandere?

Ricerca best-first

Best-first

Prima il migliore: scegliamo un nodo n che corrisponde al valore minimo di una **funzione di valutazione** $f(n)$

Best-first

Prima il migliore: scegliamo un nodo n che corrisponde al valore minimo di una **funzione di valutazione** $f(n)$

- Scegliamo un nodo sulla frontiera in cui $f(n)$ ha valore minimo.
- Restituiamo il nodo al punto prec., se lo stato e' uno stato **obiettivo**, altrimenti **generiamo i suoi nodi figli**.
- Ogni nodo figlio viene **aggiunto alla frontiera se non era stato gia' raggiunto in precedenza**, oppure **se era stato raggiunto, ma con un cammino di costo maggiore**.
- Alla fine, l'algoritmo restituisce un'indicazione di **fallimento**, oppure un **nodo che rappresenta un cammino che porta a un obiettivo**.

Best-first

Prima il migliore: scegliamo un nodo n che corrisponde al valore minimo di una **funzione di valutazione** $f(n)$

Usando funzioni di valutazione $f(n)$ diverse otteniamo algoritmi specifici diversi.

Strutture dati per la ricerca

Per tenere traccia dell'albero di ricerca

Nodo

- $n.STATO$ – Lo stato a cui corrisponde il nodo
- $n.PADRE$ – Il nodo nell'albero di ricerca che ha generato il nodo corrente
- $n.AZIONE$ – L'azione applicata allo stato del padre per generare il nodo corrente
- $n.COSTO-CAMMINO$ – Il costo totale del cammino dallo stato iniziale al nodo corrente
 - $g(n)$ in formule matematiche

Nodo

- $n.STATO$ - Lo stato a cui corrisponde il nodo
- $n.PADRE$ - Il nodo nell'albero di ricerca che ha generato il nodo corrente
- $n.AZIONE$ - L'azione applicata allo stato del padre per generare il nodo corrente
- $n.COSTO-CAMMINO$ - Il costo totale del cammino dallo stato iniziale al nodo corrente
 - $g(n)$ in formule matematiche

Seguendo i puntatori $PADRE$ a partire da un nodo, possiamo risalire lungo l'albero.

Da un nodo obiettivo possiamo ricostruire la soluzione.

Frontiera

Implementata tramite una **coda**.

Le operazioni svolte sono:

- **VUOTA?**(*frontiera*) - restituisce True solo se non ci sono piu' nodi nella frontiera
- **POP** (*frontiera*) - rimuove il nodo in cima alla frontiera e lo restituisce
- **TOP**(*frontiera*) - restituisce il nodo in cima alla frontiera senza rimuoverlo
- **AGGIUNGI**(*nodo*, *frontiera*) - inserisce un nodo al posto appropriato nella coda.

Tre tipi di code

- **coda con priorit **: estratto prima il nodo di costo minimo in base alla funzione di valutazione $f(\cdot)$
 - Usato nella ricerca **best-first**
- **coda FIFO** (first-in, first-out): estratto prima il nodo che e' stato aggiunto per primo
 - Usato nella ricerca in **ampiezza**
- **coda LIFO** (last-in, first-out): estratto prima il nodo che e' stato aggiunto per ultimo
 - Usato nella ricerca in **profondita'**

Stati raggiunti

Tabella di lookup (es. tabella hash)

- ogni chiave è uno stato
- ogni valore è il nodo per tale stato

Es.

$\text{raggiunti}(s)$ è il nodo corrispondente allo stato s

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

 aggiungi *figlio* a *frontiera*

return *fallimento*

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

 aggiungi *figlio* a *frontiera*

return *fallimento*

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

aggiungi *figlio* a *frontiera*

return *fallimento*

A ogni iterazione scegliamo un nodo sulla frontiera in cui $f(n)$ ha valore minimo

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

aggiungi *figlio* a *frontiera*

return *fallimento*

Se lo stato è uno stato
obiettivo lo restituiamo in
output

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

aggiungi *figlio* a *frontiera*

return *fallimento*

Altrimenti generiamo i figli
usando la funzione Espandi

un generatore che genera una sequenza di valori, uno ogni volta che si incontra l'espressione yield.



```
fu function ESPANDI(problema, nodo) yields nodi
    s ← nodo.STATO
    for each azione in problema.AZIONI(s) do
        s' ← problema.RISULTATO(s, azione)
        costo ← nodo.COSTO-CAMMINO + problema.COSTO-AZIONE(s, azione, s')
        yield NODO(STATO=s', PADRE=nodo, AZIONE=azione, COSTO-CAMMINO=costo)
    if problema.E-OBIETTIVO(nodo.STATO) then return nodo
    for each figlio in ESPANDI(problema, nodo) do
        s ← figlio.STATO
        if s non è in raggiunti or figlio.COSTO-CAMMINO < raggiunti[s].COSTO-CAMMINO then
            raggiunti[s] ← figlio
            aggiungi figlio a frontiera
    return fallimento
```

e
E e valore *nodo*

function RICERCA-BEST-FIRST(*problema*, *f*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(STATO=*problema*.STATOINIZIALE)

frontiera $\leftarrow$ una coda con priorità ordinata in base a *f*, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ una tabella di lookup, con un elemento con chiave *problema*.STATOINIZIALE e valore *nodo*

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *s* non è in *raggiunti* **or** *figlio*.COSTO-CAMMINO < *raggiunti*[*s*].COSTO-CAMMINO **then**

raggiunti[*s*] $\leftarrow$ *figlio*

aggiungi *figlio* a *frontiera*

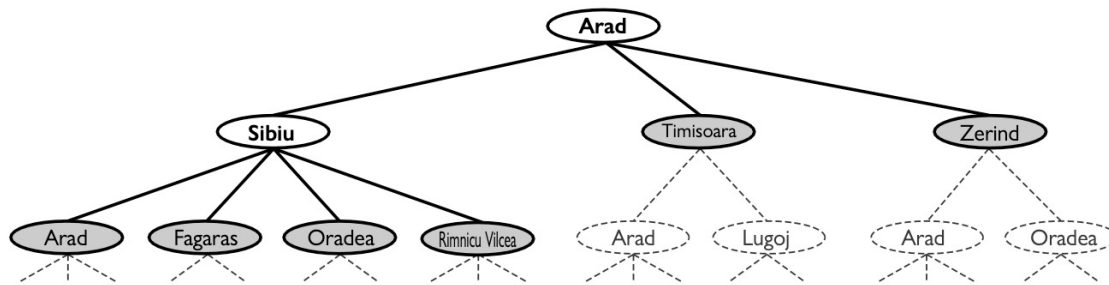
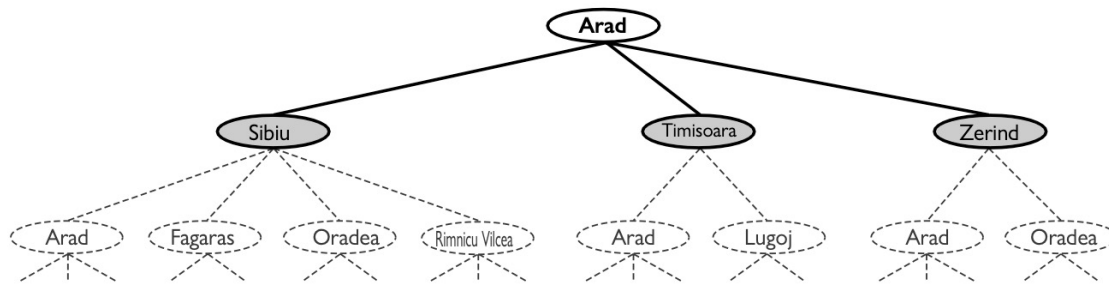
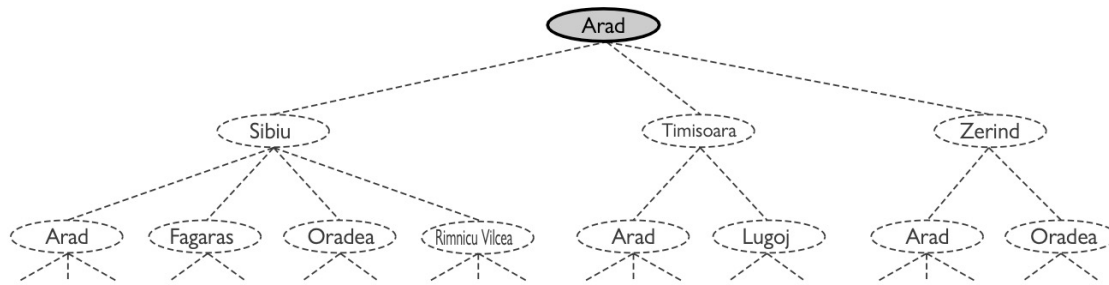
return *fallimento*

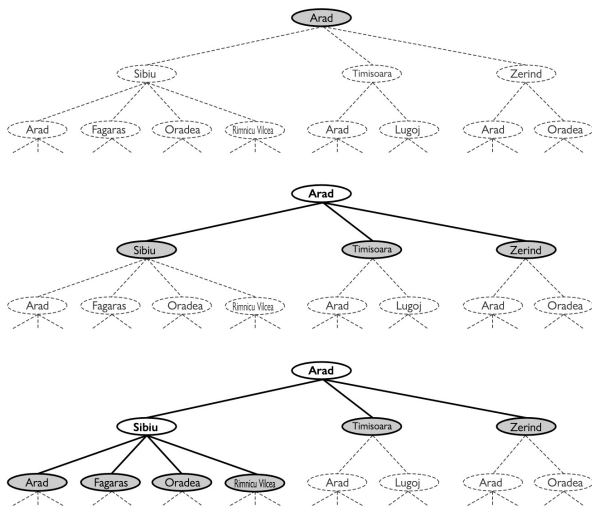
Se lo stato non è in *raggiunti*
oppure se è in *raggiunti* ma
con un costo maggiore, allora
lo inseriamo nella *frontiera*

Nota: non andremo ad
aggiungere alla *frontiera* un nodo
che corrisponde a uno stato già
raggiunto con un costo più basso.

Stato ripetuto

Stato che corrisponde a più nodi nello stesso cammino



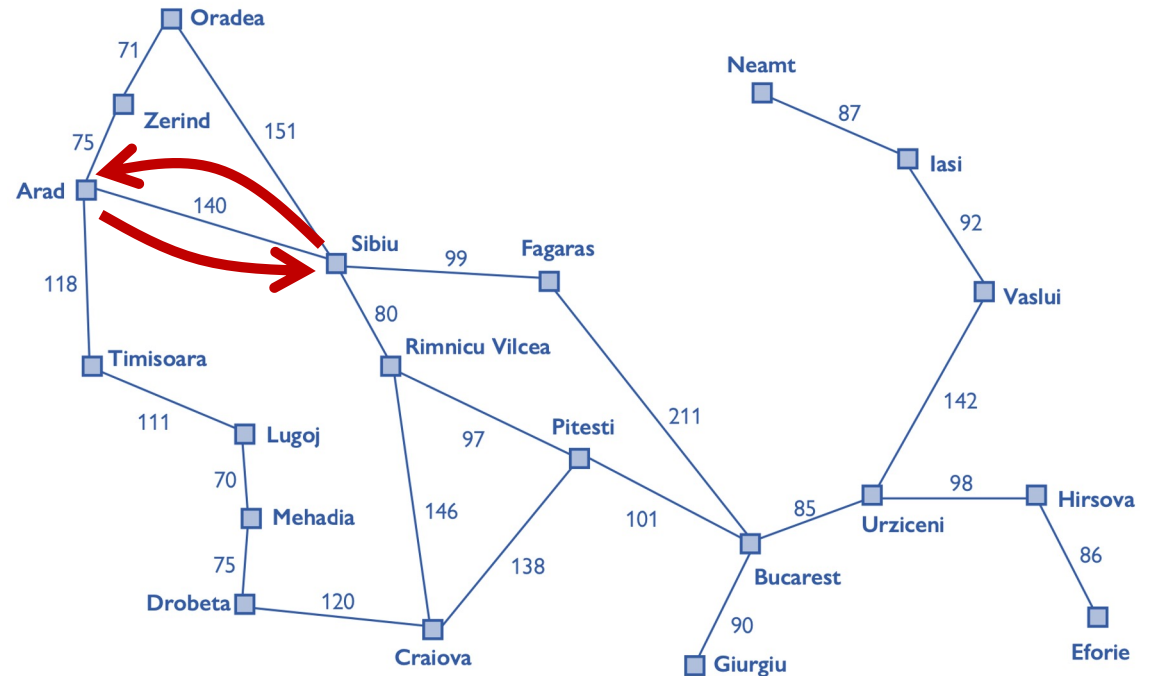


Stato ripetuto

Stato che corrisponde a più nodi nello stesso cammino

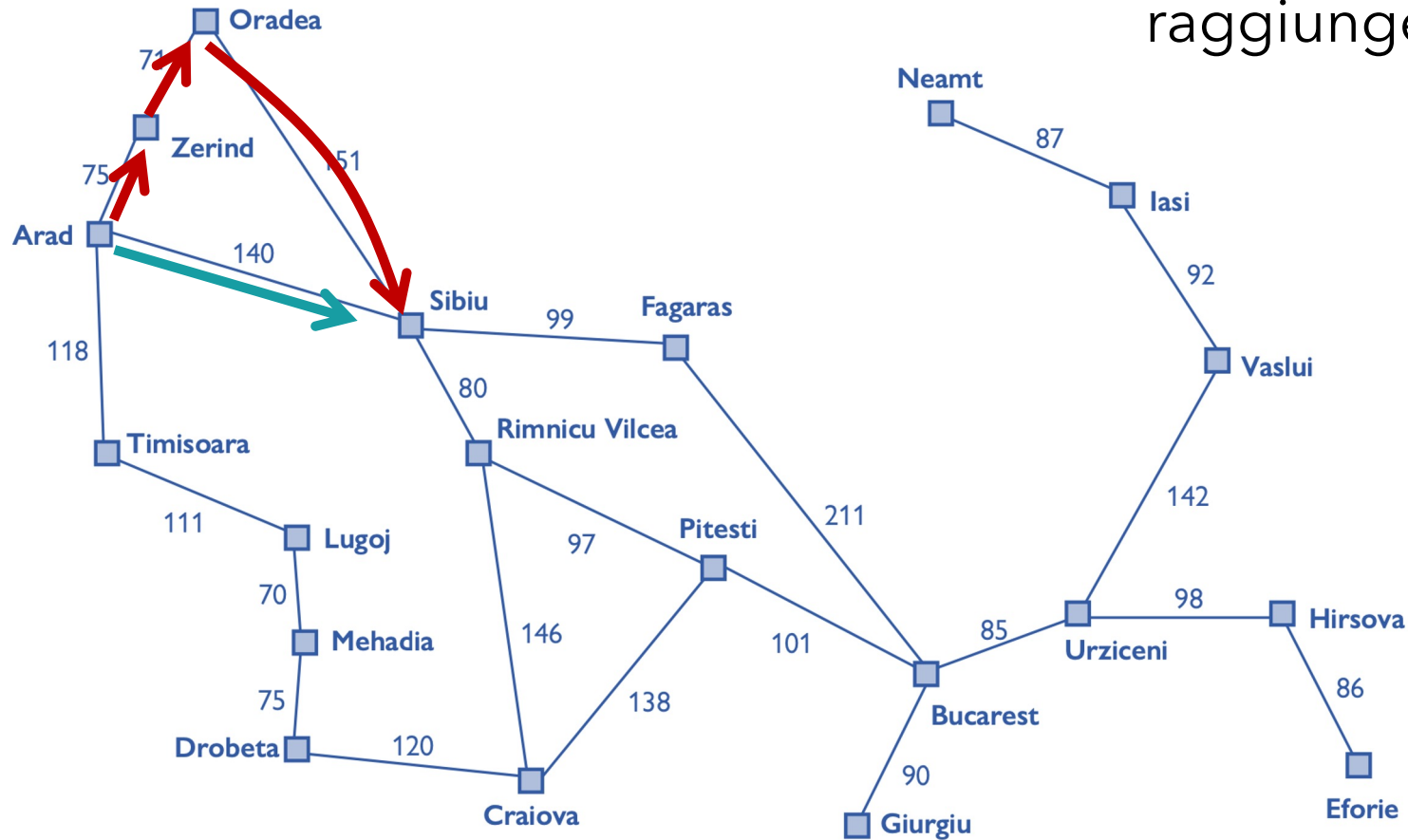
Cammino ciclico
(o ciclo)

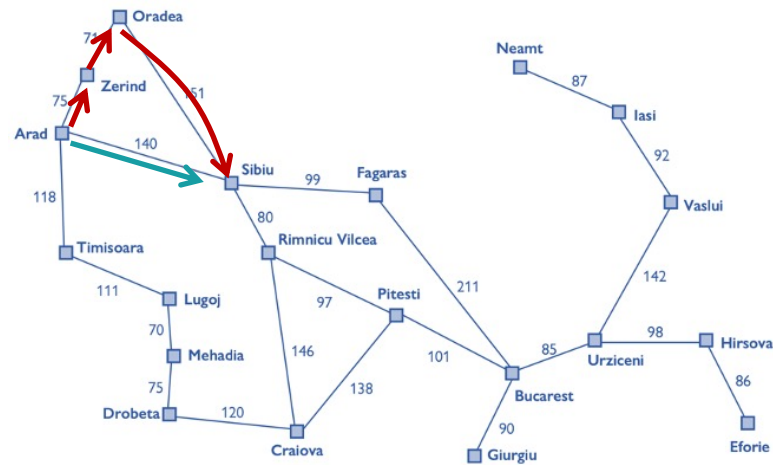
Anche se lo spazio degli stati ha un piccolo numero di stati, l'albero di ricerca completo è infinito



Cammino ridondante

Una via peggiore per raggiungere lo stesso stato





Ricerca su grafo

Controlla la presenza di cammini ridondanti.

Ricerca ad albero

Non controlla la presenza di cammini ridondanti.

Cammini ridondanti

1. Ricordare tutti gli stati precedentemente raggiunti
 - come in ricerca best-first
 - consente di individuare tutti i cammini ridondanti e mantenere solamente il cammino migliore per raggiungere ogni stato.
 - Adatto per:
 - ✓ spazio degli stati in cui ci siano molti cammini ridondanti
 - ✓ casi in cui c'è sufficiente memoria per tenere traccia dei raggiunti

Cammini ridondanti

2. Non preoccuparsi dei cammini ridondanti

- problemi in cui è raro o impossibile che due cammini diversi conducano allo stesso stato
 - ✓ es.: montaggio automatico

Cammini ridondanti

3. Controllare la presenza di cammini ciclici, ma non di cammini ridondanti

- Possiamo controllare la presenza di cicli senza aggiungere memoria, usando la catena di puntatori ai padri (all'indietro nell'albero)
- Verificare se lo stato alla fine del cammino è già apparso in precedenza
- Possibile risalire l'intero cammino, oppure solo pochi collegamenti (eliminando solo i cammini ciclici brevi).

Misure di prestazione

Valutiamo le prestazioni di un algoritmo in base a 4 parametri.

1. Completezza – l'algoritmo garantisce di trovare una soluzione, se esiste, e di riportare correttamente il fallimento, se non esiste

Per essere completo un algoritmo di ricerca deve **procedere in modo sistematico** nell'esplorazione di uno spazio degli stati infinito.

Misure di prestazione

Valutiamo le prestazioni di un algoritmo in base a 4 parametri.

1. Completezza - l'algoritmo garantisce di trovare una soluzione, se esiste, e di riportare correttamente il fallimento, se non esiste

2. Ottimalità rispetto al costo - trova la soluzione con il costo di cammino minimo?

Misure di prestazione

Valutiamo le prestazioni di un algoritmo in base a 4 parametri.

1. Completezza - l'algoritmo garantisce di trovare una soluzione, se esiste, e di riportare correttamente il fallimento, se non esiste
2. Ottimalità rispetto al costo - trova la soluzione con il costo di cammino minimo?

3. Complessità temporale - quanto tempo impiega a trovare una soluzione?

Misure di prestazione

Valutiamo le prestazioni di un algoritmo in base a 4 parametri.

1. Completezza - l'algoritmo garantisce di trovare una soluzione, se esiste, e di riportare correttamente il fallimento, se non esiste
2. Ottimalità rispetto al costo - trova la soluzione con il costo di cammino minimo?
3. Complessità temporale - quanto tempo impiega a trovare una soluzione?
4. **Complessità spaziale** - quanta memoria necessita per effettuare la ricerca?

Misure di prestazione

Valutiamo le prestazioni di un algoritmo in base a 4 parametri.

1. **Completezza** – l'algoritmo garantisce di trovare una soluzione, se esiste, e di riportare correttamente il fallimento, se non esiste
2. **Ottimalità rispetto al costo** – trova la soluzione con il costo di cammino minimo?
3. **Complessità temporale** – quanto tempo impiega a trovare una soluzione?
4. **Complessità spaziale** – quanta memoria necessita per effettuare la ricerca?

Misurare la Complessità

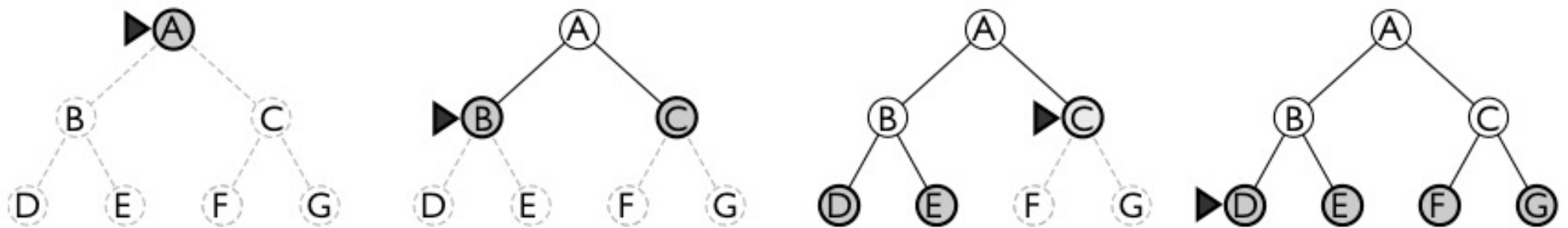
- Se il grafo è una struttura esplicita si usa come misura di difficoltà del problema $|V| + |E|$
- In pratica, misuriamo la complessità in termini di:
 - depth (profondità) d : il numero di azioni di una soluzione ottima;
 - massima profondità m : numero massimo di azioni in un qualsiasi cammino ($d \leq m$)
 - branching factor (fattore di ramificazione) b : numero di successori di un nodo

Strategie di ricerca non informata

Nessuna informazione su quanto uno stato sia vicino all'obiettivo

Ricerca in ampiezza

- Appropriata quando tutte le azioni hanno lo stesso costo
- Espandere il nodo radice, poi tutti i suoi successori, poi i successori dei successori, ecc.



Si diffonde a «ondate» di profondità uniforme

Ricerca in ampiezza

Implementazione 1:

Chiamata a Ricerca-Best-First, usando come funzione di valutazione $f(n) = \text{depth}(n)$

Ricerca in ampiezza - Implementazione 2 (più efficiente)

- Usare una coda FIFO per la frontiera
 - i nodi inseriti nella frontiera verranno espansi in ordine di inserimento
- Raggiunti può essere un insieme di stati (non una lookup table)
 - Una volta raggiunto uno stato non possiamo trovare un cammino migliore per raggiungerlo.
- Test obiettivo anticipato (invece che a posteriori)
 - Controllare se un nodo è soluzione appena viene generato, non quando viene estratto dalla coda della frontiera.

function RICERCA-IN-AMPIEZZA(*problema*) **returns** un nodo soluzione o *fallimento*

nodo $\leftarrow$ NODO(*problema*.STATOINIZIALE)

if *problema*.È-OBIETTIVO(*nodo*.STATO) **then return** *nodo*

frontiera $\leftarrow$ una coda FIFO, con *nodo* come elemento iniziale

raggiunti $\leftarrow$ {*problema*.STATOINIZIALE}

while not VUOTA?(*frontiera*) **do**

nodo $\leftarrow$ POP(*frontiera*)

for each *figlio* **in** ESPANDI(*problema*, *nodo*) **do**

s $\leftarrow$ *figlio*.STATO

if *problema*.È-OBIETTIVO(*s*) **then return** *figlio*

if *s* non è in *raggiunti* **then**

aggiungi *s* a *raggiunti*

aggiungi *figlio* a *frontiera*

return *fallimento*

Ricerca in ampiezza
Implementazione 2
(più efficiente)

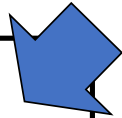
Ricerca in ampiezza: analisi

- Completa: ricerca sistematica
- Ottima rispetto al costo per problemi in cui ogni azione ha lo stesso costo
 - Quando genera nodi a profondità d , ha già generato tutti i nodi a profondità $d - 1$
- Costo temporale e spaziale
 - Numero di nodi generati in totale per soluzione a profondità d : $\sum_{i=0}^d b^i = O(b^d)$

Ricerca in ampiezza: costo

Esempio: $b = 10$; 1 milione nodi al sec generati; 1 nodo occupa 1000 byte

Piu incisivo!



Profondità	Nodi	Tempo	Memoria
2	110	0,11 ms	107 kilobyte
4	11.100	11 ms	10,6 megabyte
6	10^6	1.1 sec	1 gigabyte
8	10^8	2 min	103 gigabyte
10	10^{10}	3 ore	10 terabyte
12	10^{12}	13 giorni	1 petabyte
14	10^{14}	3,5 anni	1 esabyte

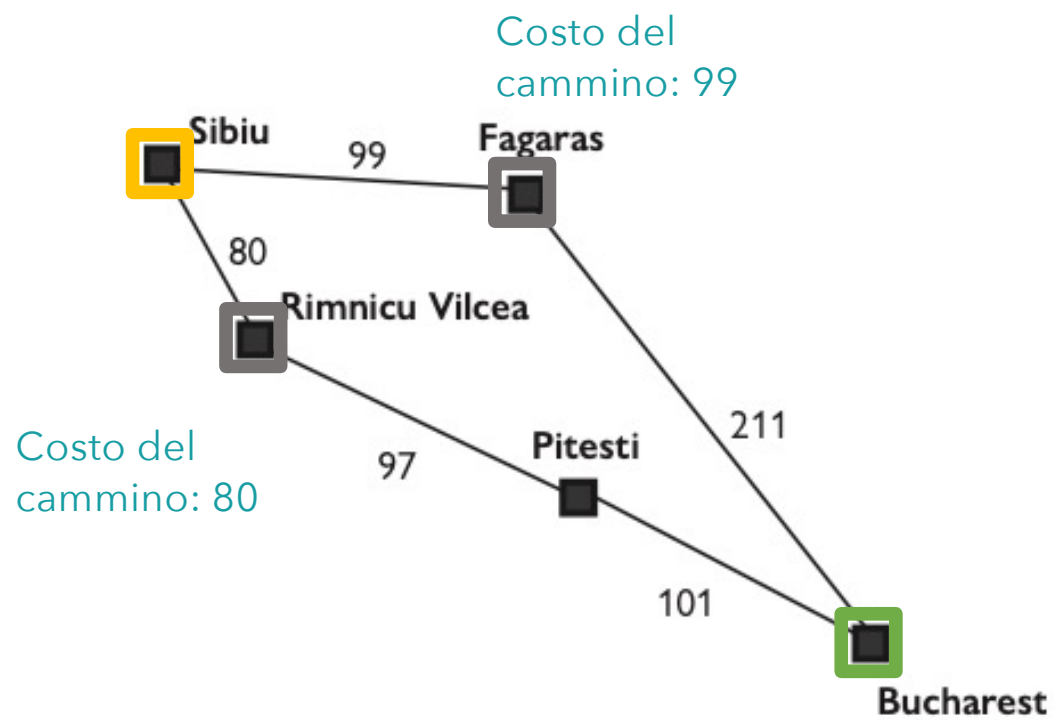
Scala male: solo istanze piccole!

Ricerca a costo uniforme (alg. di Dijkstra)

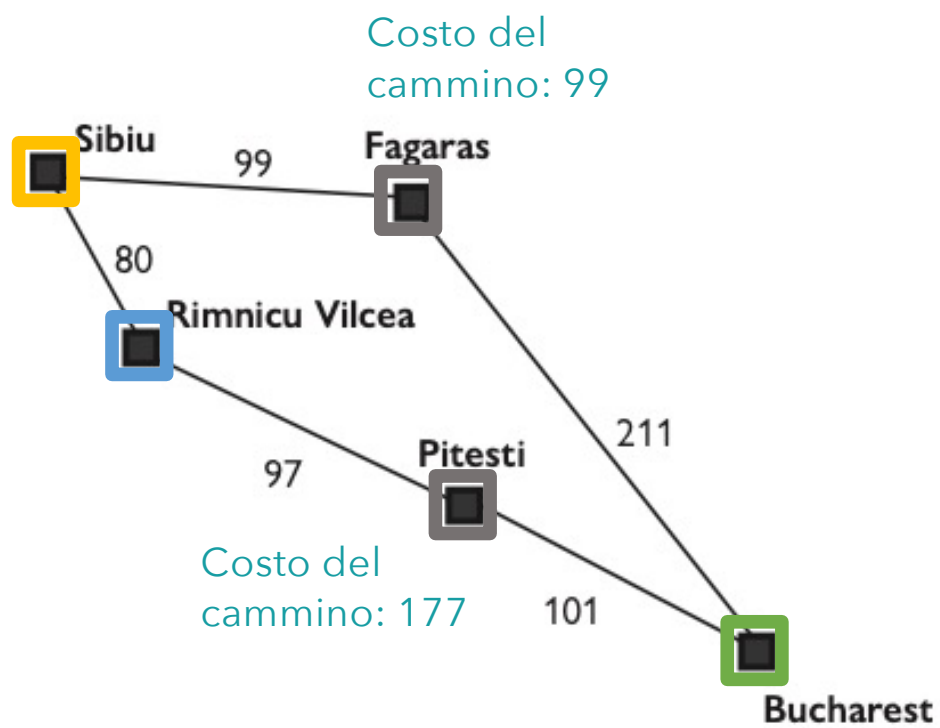
- Se le azioni hanno costi diversi
- Ricerca best-first con
funzione di valutazione = costo del cammino
dalla radice fino al nodo corrente
 - Si diffonde (propaga, sviluppa) a «ondate» di costo uniforme

function RICERCA-COSTO-UNIFORME(*problema*) **returns** un nodo soluzione o *fallimento*
return RICERCA-BEST-FIRST(*problema*, COSTO-CAMMINO)

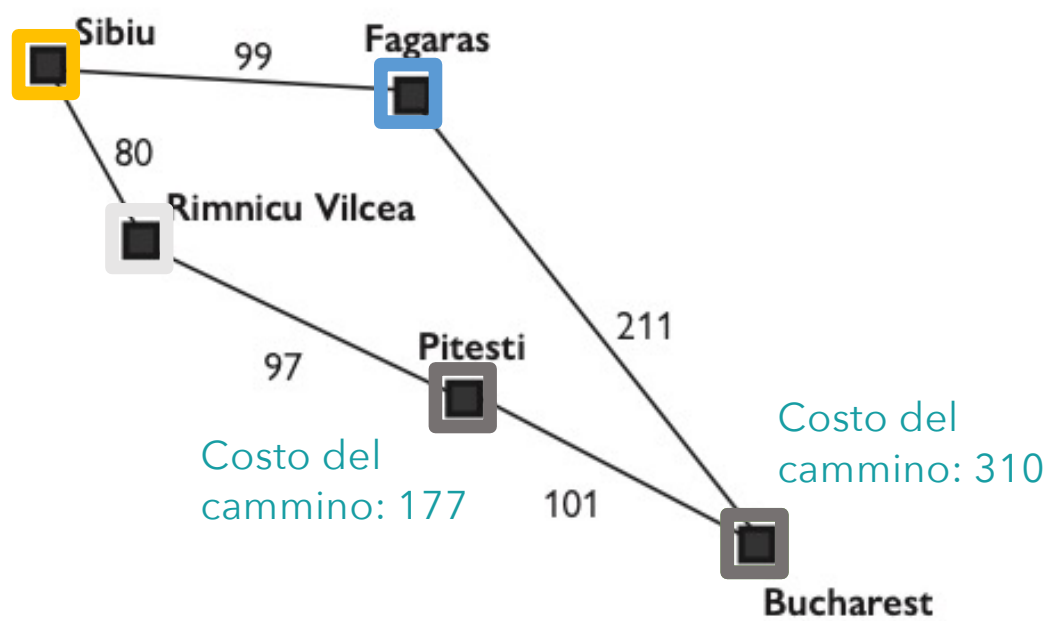
Ricerca costo uniforme



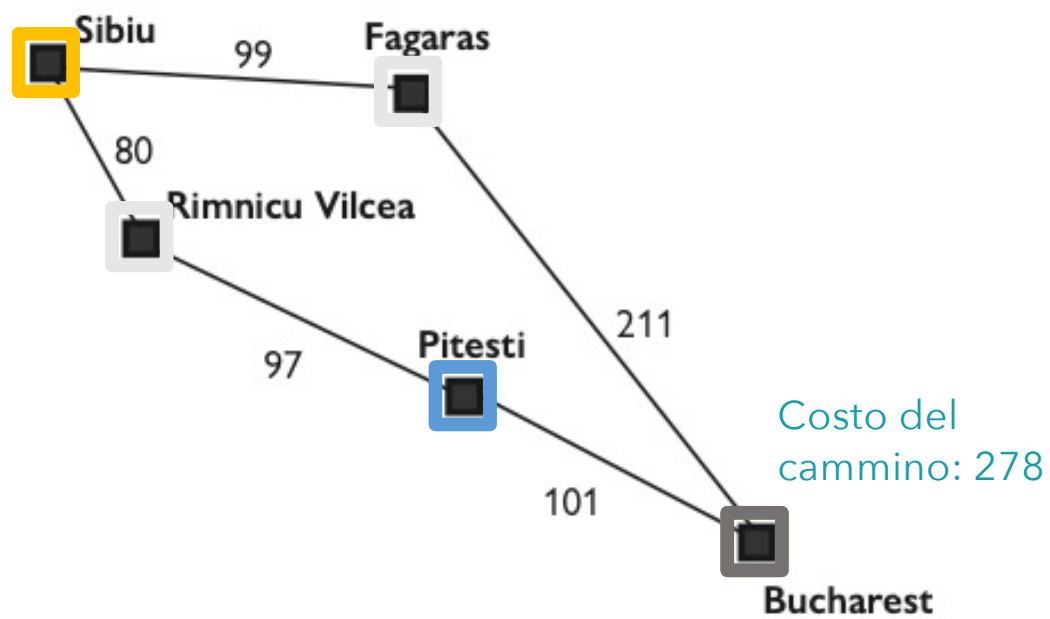
Ricerca costo uniforme



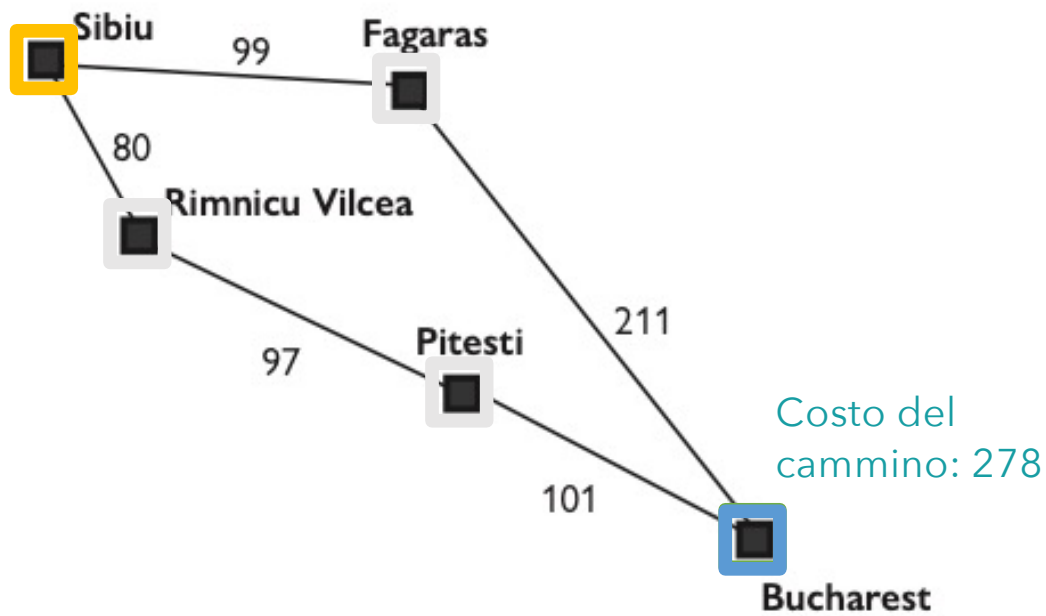
Ricerca costo uniforme



Ricerca costo uniforme

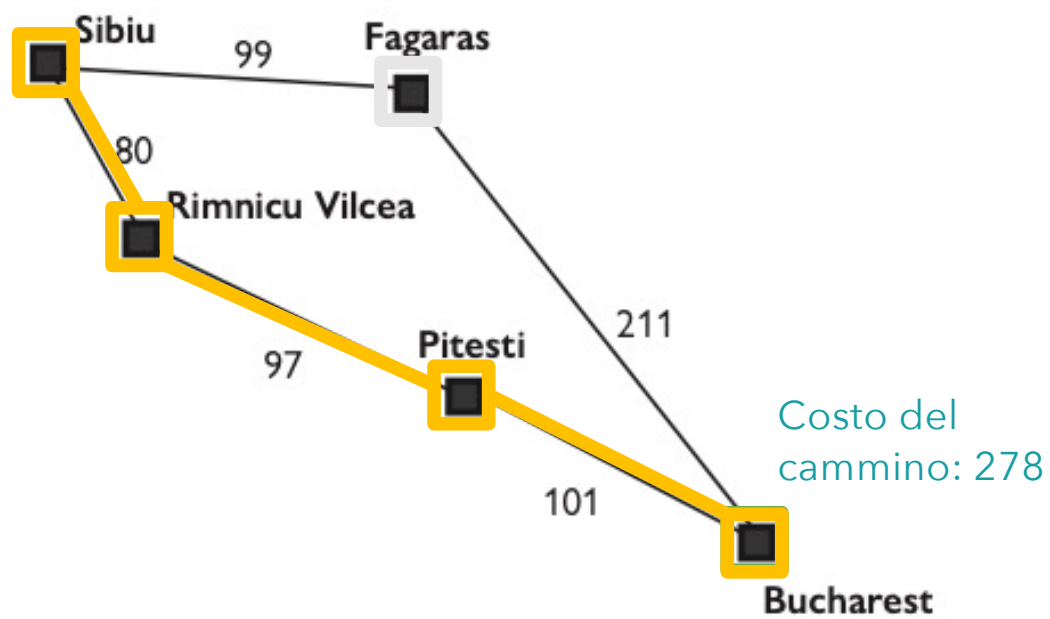


Ricerca
costo
uniforme



È-Obiettivo ha valore True

Ricerca
costo
uniforme



È-Obiettivo ha valore True

Ricerca a costo uniforme: analisi

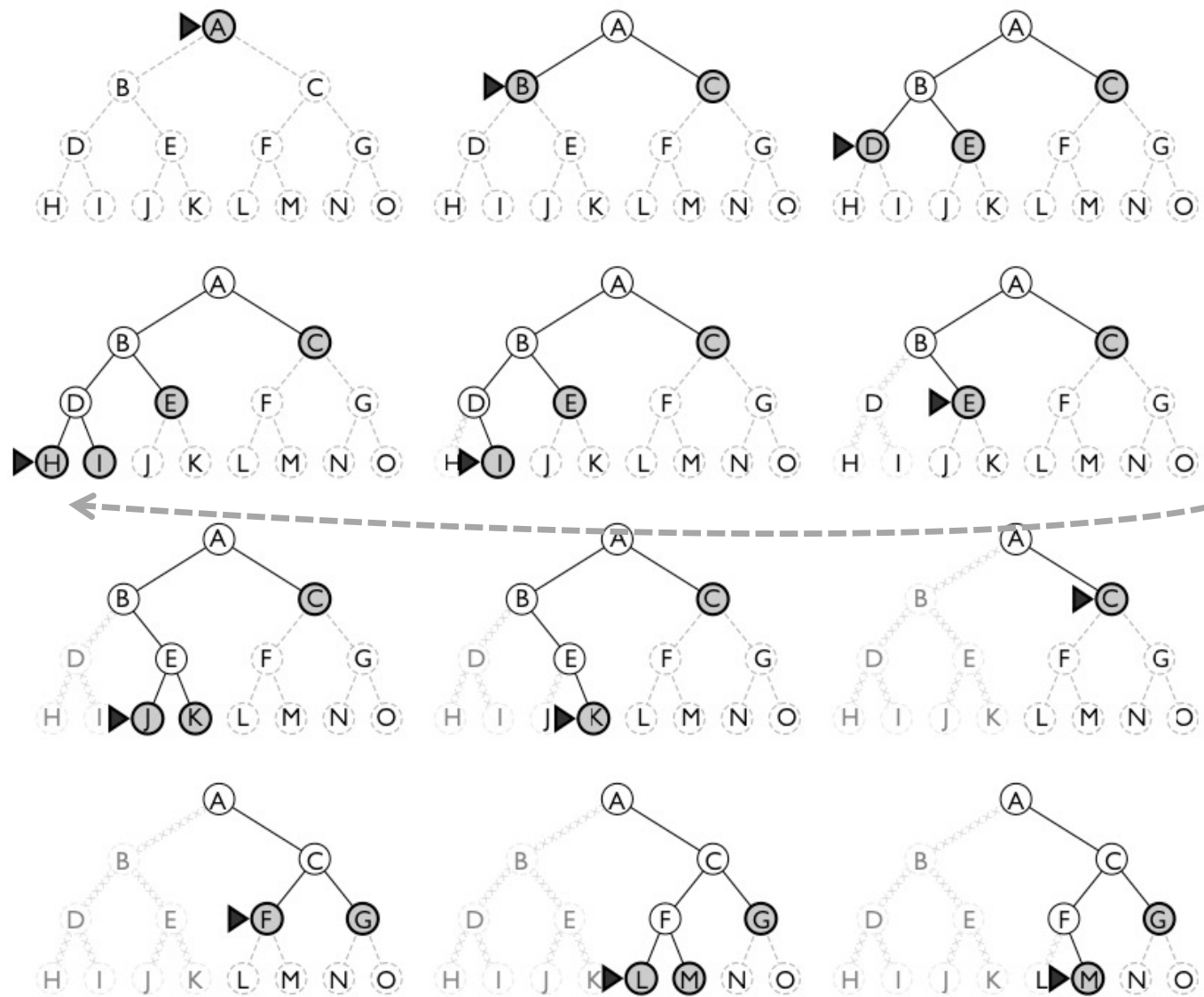
- **Completa:** ricerca sistematica
 - Esamina sistematicamente tutti i cammini in ordine di costo crescente
 - Non entra mai in un cammino infinito (assumendo che tutte le azioni abbiano un costo $> \epsilon > 0$)
- **Ottima rispetto al costo**
 - La prima soluzione che trova ha un costo basso almeno come quello di ogni altro nodo sulla frontiera

Ricerca a costo uniforme: analisi

- Complessità caratterizzata in termini di
 - C^* - costo della soluzione ottima
 - ϵ - limite inferiore imposto al costo di ogni azione
- $\left\lceil \frac{C^*}{\epsilon} \right\rceil$ è il numero maggiore possibile di azioni che servono per raggiungere il costo della soluzione ottima
 - Può essere $\gg d$
- Costo in tempo e spazio $O\left(b^{1+\left\lceil \frac{C^*}{\epsilon} \right\rceil}\right)$ goal-test e arresto posticipato, solo dopo aver generato anche l'ultima frontiera, oltre la profondità del goal
- Se ogni azione ha lo stesso costo, complessità simile a ricerca in ampiezza, ma $O(b^{1+d})$

Ricerca in profondità

- Espande il nodo a profondità maggiore nella frontiera
- Può essere implementata come una ricerca **best-first** con funzione di valutazione $f(n) = -depth(n)$
- Generalmente, implementata come ricerca ad albero senza tabella dei raggiunti
 - Usando una coda LIFO per la frontiera



Ricerca in profondità

La ricerca raggiunge immediatamente il livello più profondo dell'albero, dove i nodi non hanno successori, quindi "torna indietro" al nodo più profondo che ha ancora successori non espansi.

Ricerca in profondità: analisi

- Per spazi degli stati finiti e che sono alberi, è completa ed efficiente
- Per spazi degli stati aciclici, può arrivare ad espandere lo stesso stato più volte attraverso cammini diversi, ma resta sistematica
- Per spazi degli stati ciclici, può bloccarsi in un ciclo infinito
- Per spazi degli stati infiniti la ricerca in profondità non è sistematica: **incompleta!**

Ricerca in profondità: analisi

- PRO: quando è praticabile una ricerca ad albero è molto efficiente (in memoria).
 - Non viene mantenuta una tabella di raggiunti
- Se spazio degli stati finito e a forma di albero
 - Richiede tempo proporzionale al numero di stati: $O(b^m)$
 - Richiede memoria solo $O(bm)$ Ho b nodi generati per ciascuno degli m livelli di profondità dell'albero
 - Backtracking: quando si espande un nodo viene generato solo un successore e si ricorda quale deve essere generato in seguito
 - ✓ solo $O(m)$

Ricerca a profondità limitata

- Forniamo un **limite l** di profondità **alla** ricerca in **profondità** per evitare che si perda in un cammino infinito
 - Consideriamo i nodi a profondità $> l$ come se non avessero successori
- Come scegliere l ?
 - In base alla conoscenza del problema
 - Diametro dello spazio degli stati

Ricerca a profondità limitata

```
function RICERCA-PROFONDITÀ-LIMITATA(problema,  $\ell$ ) returns un nodo soluzione o fallimento o soglia  
  frontiera  $\leftarrow$  una coda LIFO (stack) con NODO(problema.STATOINIZIALE) come elemento iniziale  
  risultato  $\leftarrow$  fallimento  
  while not VUOTA?(frontiera) do  
    nodo  $\leftarrow$  POP(frontiera)  
    if problema.È-OBIETTIVO(nodo.STATO) then return nodo  
    if PROFONDITÀ(nodo) >  $\ell$  then  
      risultato  $\leftarrow$  soglia  
    else if not È-CICLO(nodo) do  
      for each figlio in ESPANDI(problema, nodo) do  
        aggiungi figlio a frontiera  
  return risultato
```

Soglia: valore di output che segnala che potrebbe esistere una soluzione a un livello di profondità maggiore di ℓ

Ricerca a profondità limitata

- Se si sceglie male l , l'algoritmo non riesce a trovare la soluzione: **incompleto**
- **Non ottimale**
- **Complessità temporale $O(b^l)$**
- **Complessità spaziale $O(bl)$**

Ricerca ad approfondimento iterativo

Proviamo tutti i valori di l

- $l = 0, l = 1, l = 2, \text{etc} \dots$
- finché non si trova una soluzione oppure la ricerca restituisce *fallimento* (invece che *soglia*)

```
function RICERCA-APPROFONDIMENTO-ITERATIVO(problema) returns un nodo soluzione o fallimento  
  for profondità = 0 to  $\infty$  do  
    risultato  $\leftarrow$  RICERCA-PROFONDITÀ-LIMITATA(problema, profondità)  
    if risultato  $\neq$  soglia then return risultato
```

Ricerca ad approfondimento iterativo

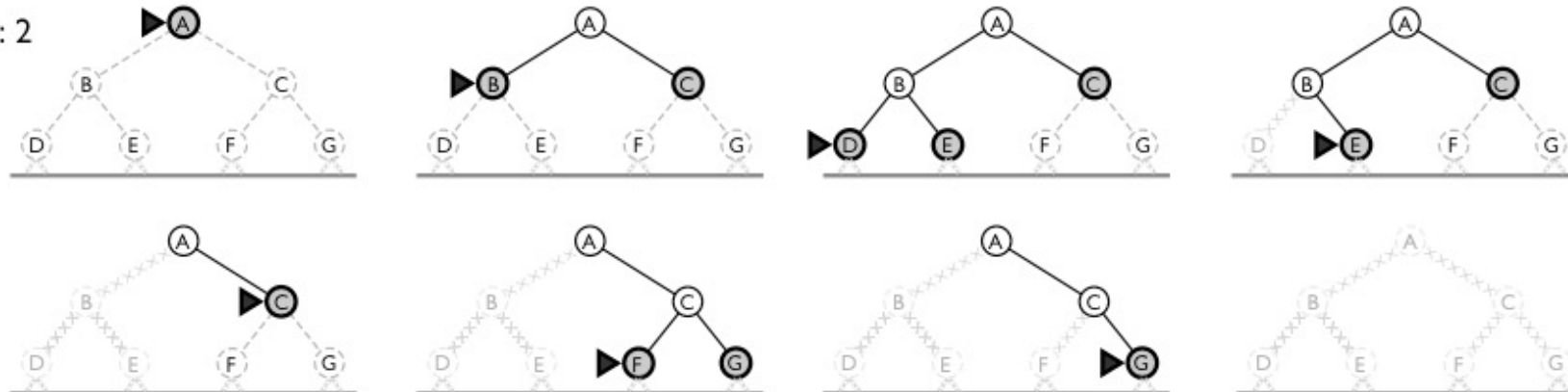
limite: 0



limite: 1

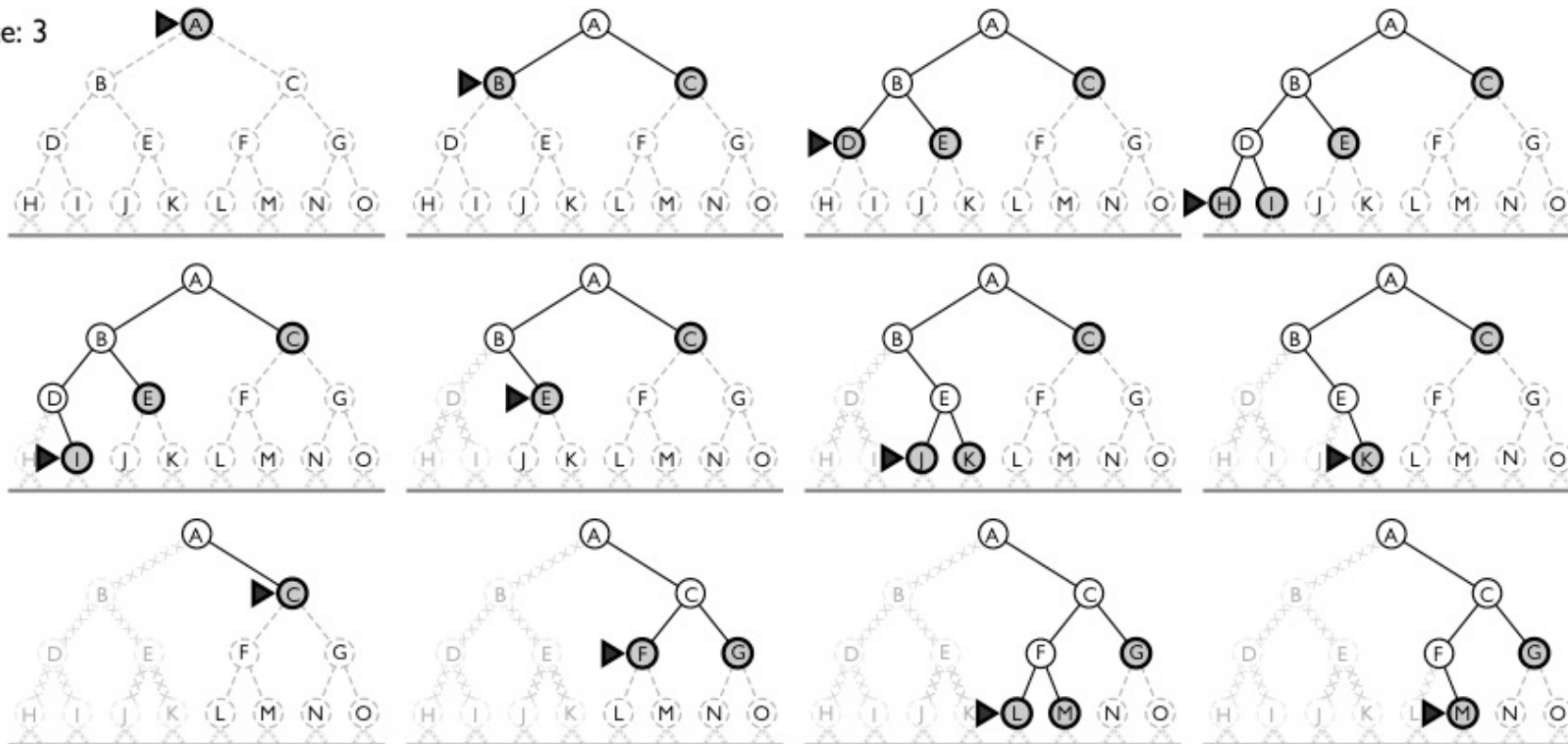


limite: 2



Ricerca ad approfondimento iterativo

limite: 3



Ricerca ad approfondimento iterativo: analisi

- Completa su spazi di stati aciclici finiti, o su qualsiasi spazio degli stati finito se controlliamo la presenza di cicli risalendo l'intero cammino
- Ottima per problemi in cui tutte le azioni hanno lo stesso costo
- Complessità temporale: $O(b^d)$ se esiste una soluzione, $O(b^m)$ se non esiste
- Complessità spaziale: $O(bd)$ se esiste una soluzione, $O(bm)$ se non esiste

Direzione della ricerca

Un problema ortogonale alla strategia è la *direzione della ricerca*:

- ricerca *in avanti* o *guidata dai dati*: si esplora lo spazio di ricerca dallo stato iniziale allo stato obiettivo;
- ricerca *all'indietro* o *guidata dall'obiettivo*: si esplora lo spazio di ricerca a partire da uno stato goal e riconducendosi a sotto-goal fino a trovare uno stato iniziale.

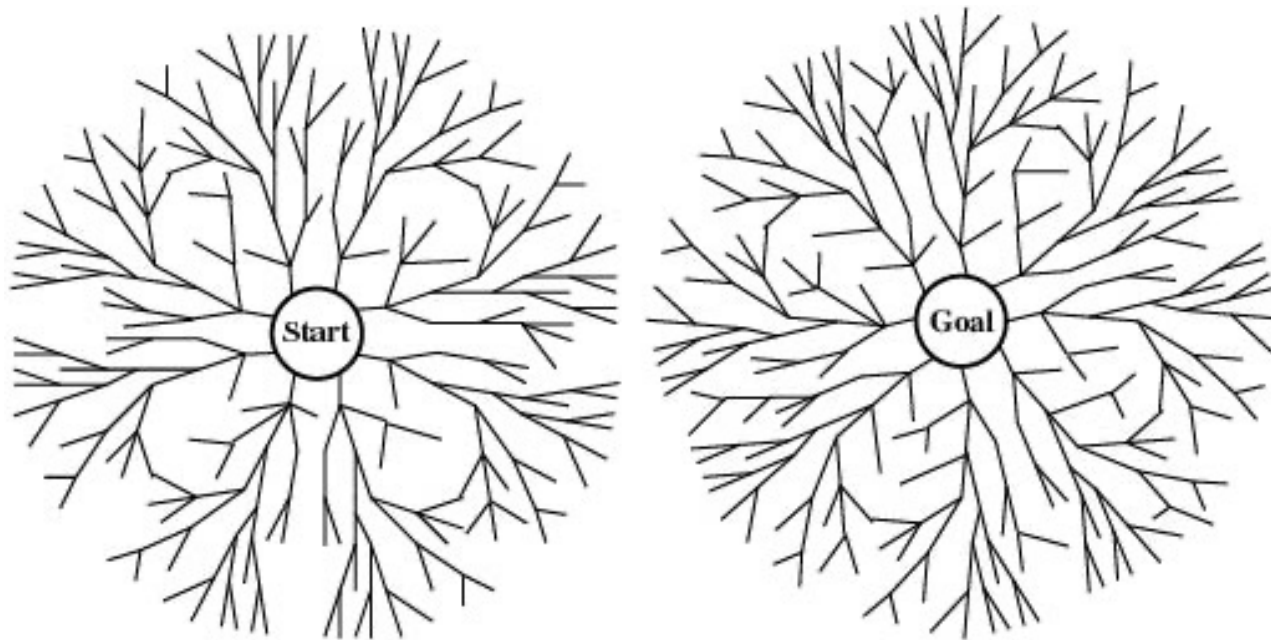
Quale direzione?

Conviene procedere nella direzione in cui il fattore di diramazione è minore

- Si preferisce ricerca all'indietro quando, e.g.:
 - l'obiettivo è chiaramente definito (e.g. theorem proving) o si possono formulare una serie limitata di ipotesi;
- Si preferisce ricerca in avanti quando, e.g.:
 - gli obiettivi possibili sono molti (design)

Ricerca bidirezionale

Si procede nelle due direzioni fino ad incontrarsi



Ricerca bidirezionale

- Occorre tenere traccia di **due frontiere** e avere **due tabelle di raggiunti** (es. best first bidirez.)
- **Nodo da espandere**: sempre quello con valore minimo della funzione di valutazione
 - non due nodi contemporaneamente, un solo nodo in una delle due frontiere
- Passiamo all'algoritmo due versioni del problema e della funzione di valutazione (F , B)

Ricerca bidirezionale

La prima soluzione trovata non è necessariamente la migliore, la funzione Terminato determina quando fermare la ricerca di nuove soluzioni

function RICERCA-BEST-FIRST-BIDIREZIONALE($problema_F, f_F, problema_B, f_B$) **returns** un nodo soluzione o *fallimento*

$nodo_F \leftarrow \text{NODO}(problema_F.STATOINIZIALE)$ // Nodo per uno stato iniziale

$nodo_B \leftarrow \text{NODO}(problema_B.STATOINIZIALE)$ // Nodo per uno stato obiettivo

$frontiera_F \leftarrow$ coda con priorità ordinata in base a f_F , con $nodo_F$ come elemento iniziale

$frontiera_B \leftarrow$ coda con priorità ordinata in base a f_B , con $nodo_B$ come elemento iniziale

$raggiunti_F \leftarrow$ tabella di lookup, con un elemento con chiave $nodo_F.STATO$ e valore $nodo_F$

$raggiunti_B \leftarrow$ tabella di lookup, con un elemento con chiave $nodo_B.STATO$ e valore $nodo_B$

$soluzione \leftarrow$ *fallimento*

while not TERMINATO($soluzione, frontiera_F, frontiera_B$) **do**

if $f_F(\text{TOP}(frontiera_F)) < f_B(\text{TOP}(frontiera_B))$ **then**

$soluzione \leftarrow \text{PROCEDI}(F, problema_F, frontiera_F, raggiunti_F, raggiunti_B, soluzione)$

else $soluzione \leftarrow \text{PROCEDI}(B, problema_B, frontiera_B, raggiunti_B, raggiunti_F, soluzione)$

return $soluzione$

Ricerca bidirezionale

```
function RICERCA-BEST
  soluzione o fallimento
   $nodo_F \leftarrow NODO(problema_F)$ 
   $nodo_B \leftarrow NODO(problema_B)$ 
   $frontiera_F \leftarrow$  coda c
   $frontiera_B \leftarrow$  coda c
   $raggiunti_F \leftarrow$  tabella
   $raggiunti_B \leftarrow$  tabella
   $soluzione \leftarrow$  fallime
  while not TERMINA
```

```
function PROCEDI(dir, problema, frontiera, raggiunti, raggiunti2, soluzione) returns una soluzione
  // Espande nodo su frontiera; controlla rispetto all'altra frontiera in raggiunti2.
  // La variabile "dir" è la direzione: F per avanti o B per indietro.
  nodo  $\leftarrow$  POP(frontiera)
  for each figlio in ESPANDI(problema, nodo) do
    s  $\leftarrow$  figlio.STATO
    if s not in raggiunti or COSTO-CAMMINO(figlio) < COSTO-CAMMINO(raggiunti[s]) then
      raggiunti[s]  $\leftarrow$  figlio
      aggiungi figlio a frontiera
      if s è in raggiunti2 then
        soluzione2  $\leftarrow$  UNISCI-NODI(dir, figlio, raggiunti2[s]))
        if COSTO-CAMMINO(soluzione2) < COSTO-CAMMINO(soluzione) then
          soluzione  $\leftarrow$  soluzione2
  return soluzione
```

```
  if  $f_F(TOP(frontiera_F)) < f_B(TOP(frontiera_B))$  then
    soluzione  $\leftarrow$  PROCEDI(F, problemaF, frontieraF, raggiuntiF, raggiuntiB, soluzione)
  else soluzione  $\leftarrow$  PROCEDI(B, problemaB, frontieraB, raggiuntiB, raggiuntiF, soluzione)
```

```
return soluzione
```

Ricerca bidirezionale: analisi

- Complessità tempo: $O(b^{d/2})$ [/2 = radice quadrata!]
(assumendo test intersezione in tempo costante, es. hash table)
- Complessità spazio: $O(b^{d/2})$
(almeno tutti i nodi in una direzione in memoria, es. usando BF)

NOTA: non sempre applicabile, es. predecessori non definiti, troppi stati obiettivo ...

Confronto

b – branching factor m – massima profondità albero di ricerca
 d – profondità della soluzione ottima l – limite alla profondità

criterio	in ampiezza	a costo uniforme	in profondità	a profondità limitata	ad approfondimento iterativo	bidirezionale (se applicabile)
completa?	Sì ¹	Sì ^{1,2}	No	No	Sì ¹	Sì ^{1,4}
ottima rispetto al costo?	Sì ³	Sì	No	No	Sì ³	Sì ^{3,4}
tempo	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
spazio	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$

- ¹ – se b è finito, e lo spazio degli stati ha una soluzione o è finito
- ² – completa se tutti i costi di azione sono $\geq \varepsilon > 0$
- ³ – ottima rispetto al costo se i costi delle azioni sono tutti identici
- ⁴ – se in entrambe le direzioni si usa una ricerca in ampiezza o una ricerca a costo uniforme

Conclusioni

- Gli algoritmi di ricerca sono valutati sulla base della loro completezza, ottimalità, complessità temporale e complessità spaziale.
- I metodi di ricerca non informata hanno accesso soltanto alla definizione del problema.
- Gli algoritmi costruiscono un albero di ricerca nel tentativo di trovare una soluzione, e differiscono tra loro in base a quale nodo espandono prima

Conclusioni

- La ricerca **best-first** seleziona i nodi per l'espansione usando una **funzione di valutazione**.
- La ricerca **in ampiezza** espande **prima i nodi più vicini alla radice**
 - Completa; ottima quando le azioni hanno costo unitario, ma ha una complessità spaziale esponenziale.
- La ricerca a **costo uniforme** espande il **nodo che ha minor costo di cammino, $g(n)$**
 - ottima per costi di passo qualsiasi.

Conclusioni

- La ricerca **in profondità** espande prima **il nodo più profondo** non ancora espanso.
 - Non è completa né ottima, ma ha complessità spaziale lineare.
 - La ricerca a **profondità limitata** prevede un limite alla profondità.

Conclusioni

- La ricerca ad **approfondimento iterativo** esegue una serie di ricerche a profondità limitata, estendendo progressivamente il limite di profondità finché non trova una soluzione.
 - È completa quando viene effettuato un controllo completo di tutti i cicli, ottima quando le azioni hanno costo unitario, ha complessità temporale comparabile alla ricerca in ampiezza e complessità spaziale lineare.
- La ricerca **bidirezionale** espande due frontiere, una attorno allo stato iniziale e una attorno all'obiettivo, fermandosi quando le due frontiere entrano in contatto.